

DROS: A Four-Layer Runtime Substrate with Deterministic Execution Enforcement for Agent-to-Execution Attribution Governance

Chun-Cheng (Jimmy) Chen
Top-Celestial Company Ltd.
 Taipei, Taiwan R.O.C.
 jimmychen@dr-os.io

Abstract—Autonomous AI agents increasingly operate with legitimate credentials, delegated capabilities, and access to consequential tools, creating an attribution gap between high-level agent intent and low-level physical execution. We present DROS, a four-layer runtime substrate that separates agent identity from execution authority and enforces capability constraints at a binary C-ABI boundary. Rather than treating security as a priori established, we formulate DROS as an experimentally falsifiable execution-governance boundary and introduce a progressive adversarial validation methodology combining autonomous attack search, negative controls, mutation testing, independent ground-truth oracles, concurrency stress, and open reproducibility. The evaluation decomposes the causal chain into intent, authorization, execution, and physical effect, allowing enforcement decisions to be independently cross-validated against kernel and external state observations. Across the instantiated evaluation corpus, including adaptive attack attempts, multi-agent delegation chains, revocation races, mutation tests, and oracle probes, we observe no unauthorized physical effects within the explicitly instrumented observation boundary. We report the resulting guarantees as empirical invariants over the evaluated state space rather than universal security proofs, and release the evaluation harness and counterexample protocol to enable continued adversarial falsification.

Index Terms—AI Agent Security, Runtime Execution Substrate, Controlled C-ABI Authorization Boundary, Autonomous Adversarial Evaluation, RCU State Transition, Information Flow Control (IFC), Attribution Governance.

I. INTRODUCTION & THE SEMANTIC-KERNEL PARADOX

Autonomous AI Agents operate with valid credentials, OAuth session tokens, and database connections. In authenticated application-layer abuse scenarios, traditional perimeter controls alone are insufficient because malicious actions originate from authenticated internal principals.

A. The Semantic-Kernel Paradox

Modern systems literature identifies a fundamental split in AI defense:

- 1) **Semantic Application Middleware (High Semantics, Zero Determinism)**: Prompt firewalls, JSON schema validators, and application middlewares inspect natural language and declared schemas. However, they lack execution-boundary containment; interpreter escapes,

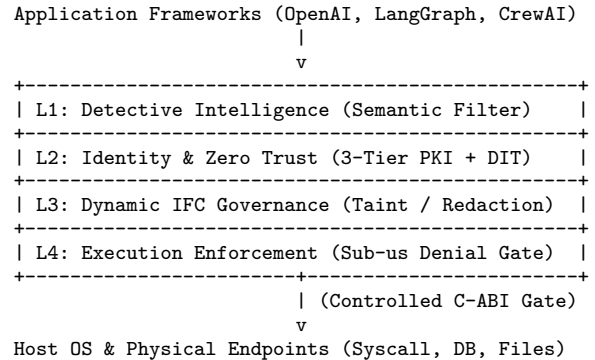


Fig. 1. DROS 4-Layer Execution-Governance Substrate Topology.

undeclared tool invocations, and semantic obfuscation consistently bypass these layers.

- 2) **Kernel Sandboxes (High Determinism, Zero Semantics)**: Low-level primitives (e.g., Seccomp, Linux Namespaces, eBPF) enforce binary syscall filtering but cannot distinguish whether a database write originated from an authorized Finance Agent or a hijacked Support Agent operating in the same worker process.

B. DROS 4-Layer Substrate Placement & Core Design Thesis

DROS resides between application frameworks and the host operating system, establishing four deep enforcement layers where **L1 may fail, but L2–L4 deterministically enforce**:

C. Core Contribution Thesis & Research Questions

DROS does not assert that “absolute security is proven a priori”; rather, it establishes an experimentally falsifiable execution-governance boundary evaluated through progressively adversarial, independently observed, and openly falsifiable experiments. We investigate four primary research questions:

- **RQ1 (Enforcement Effectiveness)**: Under post-compromise application conditions ($C_A = 1$), can the governed execution path deterministically prevent unauthorized operations from causing physical state transitions?

- **RQ2 (Revocation Atomicity):** Under concurrent execution and policy revocation races, does there exist any stale authorization window allowing replayed executions?
- **RQ3 (Evaluator Sensitivity):** Can the evaluation framework and test harness effectively detect deliberately injected security flaws and mutation regressions?
- **RQ4 (Oracle Independence):** Can authorization decisions be objectively and independently verified by decoupled execution-side and physical-side observers?

II. THREAT MODEL, EVALUATION SPACE, AND EPISTEMIC BOUNDARIES

A. Post-Compromise Threat Model ($C_A = 1$)

Our evaluation establishes a rigorous Post-Compromise threat model:

- **Assumption 1 (Full Application Compromise $C_A = 1$):** The adversary has achieved prompt injection, extracted system prompts, and holds arbitrary code execution capabilities within the application container.
- **Assumption 2 (Valid Credential Possession):** The adversary possesses legitimate OAuth tokens, internal API keys, and authenticated session identities.
- **Assumption 3 (Multi-Agent Collusion & Delegation):** The adversary controls multiple heterogeneous agents attempting privilege escalation via delegation chains and confused-deputy patterns.
- **Root of Trust:** The controlled binary C-ABI execution gate and underlying OS memory isolation.

B. Comparative Baselines & Attack Preservation

To eliminate evaluation bias from payload variance, comparative experiments strictly adhere to the Attack Preservation Principle ($A_{B0} = A_{B1}$ and $\text{Env}_{B0} \approx \text{Env}_{B1}$), as shown in Table I.

C. Formal Physical Effect & Security Invariants

Terminology Consistency:

- **Counterexample Search:** Proactive exploration process aimed at discovering executable violating states ($C_A \wedge \neg C_E \implies I_{\text{physical}} > 0$).
- **Counterexample Submission:** Public protocol enabling external researchers to submit reproducible counterexample traces to the community registry.

Physical Effect Definition: Physical effect $I_{\text{physical}}(x)$ is strictly defined over the explicitly instrumented observation set $\mathcal{S}_{\text{obs}}$:

$$I_{\text{physical}}(x) = \begin{cases} 1, & \exists s \in \mathcal{S}_{\text{obs}} : \Delta(s) > 0 \\ 0, & \forall s \in \mathcal{S}_{\text{obs}} : \Delta(s) = 0 \end{cases} \quad (1)$$

where $\mathcal{S}_{\text{obs}} = \{\text{Syscall}, \text{FS Diff}, \text{DB WAL}, \text{Process}, \text{Net}, \text{IPC}\}$.

Ghost Syscall Definition: A ghost syscall is an authorization-denied operation that nevertheless escapes to the host kernel:

$$G = \#\{\text{syscall} \mid \text{Auth} = \text{DENY} \wedge \text{syscall_observed} = 1\} = 0 \quad (2)$$

Formal Security Invariants:

$$C_A \not\Rightarrow C_E \quad (\text{Non-Inheritance of Authority}) \quad (3)$$

$$C_E(t_0) \not\Rightarrow C_E(t_1) \quad (\forall t_0 \neq t_1, \text{Temporal Isolation}) \quad (4)$$

$$\bigcup_{i=1}^n C_{A_i} \not\Rightarrow C_E^{\text{unauth}} \quad (\text{Capability Composition Safety}) \quad (5)$$

$$\forall x \in X_{\text{evaluated}}, \quad C_A(x) \wedge \neg C_E(x) \implies I_{\text{physical}}(x) = 0 \quad (6)$$

$$S_{\text{after}} \equiv S_{\text{before}} \quad (\text{Zero State Drift within } \mathcal{S}_{\text{obs}}) \quad (7)$$

III. DROS 4-LAYER SUBSTRATE IMPLEMENTATION

A. L1: Detective Intelligence (Detect)

L1 applies heuristic pattern matching and lightweight semantic filters. In DROS, **L1 is explicitly treated as probabilistic**. Threat containment falls deterministically onto L2–L4.

B. L2: Identity & Dynamic Intent Tokens (Attribute)

L2 establishes a cryptographic 3-Tier PKI attribution model (Enterprise CA $\rightarrow$ Task Issuer $\rightarrow$ Ephemeral Worker Agent) and issues Dynamic Intent Tokens (DIT). The DIT cryptographically binds the tool name, argument hash, timestamp, and capability bitmask using Ed25519 signatures.

C. L3: Dynamic IFC & In-Band Redaction (Constrain Data)

L3 implements in-memory taint tracking and in-band secret masking. When confidential assets pass through the agent context, L3 redacts them in-band (substituting [REDACTED_BY_DROS_POLICY_GATE]).

D. L4: Binary C-ABI Authorization Execution Gate (Enforce Execution)

L4 serves as the controlled binary authorization boundary, implemented in Rust and exposed via a pure C-ABI dynamic library.

- 1) **$O(1)$ Policy Lookup:** Bitmask evaluation executes in sub-microsecond bounds.
- 2) **Atomic Policy-State Transition ($T_{\text{swap}} = 420 \text{ ns}$):** Capability pointers are swapped atomically via AtomicPtr, ensuring subsequent governed calls cannot access stale authorization entries.
- 3) **Sub-Microsecond Fail-Closed Denial Path ($< 500 \text{ ns}$):** Any bitmask mismatch or unauthorized capability triggers an immediate fail-closed denial branch.

IV. PROGRESSIVE ADVERSARIAL VALIDATION FRAMEWORK

We construct a 7-tier progressive falsification hierarchy, summarized in Table II.

TABLE I
COMPARATIVE BASELINES CONFIGURATION & EVALUATION TOPOLOGIES

Baseline ID	Topology Structure	Primary Evaluation Purpose
B0 (Bare Application)	Attacker → Application → OS	Establishes unmanaged baseline; verifies exploit payloads cause real damage ($\Delta S_{B0} > 0$).
B1 (DROS Enabled)	Attacker → Application → DROS/PGM → OS	Primary subject under test; verifies execution containment ($\Delta S_{B1} \equiv 0$).
B2 (Defense-in-Depth)	Attacker → App → DROS + Stack → OS	Verifies compatibility and coverage with existing EDR/XDR/WAF infrastructure.
B3 (Pure Binary C-ABI)	Attacker → Standalone PGM C-ABI → OS	Evaluates C-ABI execution gate in complete isolation without application assistance.

TABLE II
ADVERSARIAL VALIDATION HIERARCHY, WORKLOAD, AND EMPIRICAL METRICS

Validation Tier	Methodology & Configuration	Adversarial Workload	Observed Defense / Evidence	CE	Verdict
L1 Baseline	B0/B1 A/B removal comparison	50 unmanaged exploits	B0 breached ($\Delta S_{B0} > 0$); B1 contained ($\Delta S_{B1} \equiv 0$)	0	PASS (RQ1)
L2 Autonomous	Strix white-box adaptive search	2,410 attempts (21.4M Tokens)	100% of probes reaching gateway rejected under corpus	0	PASS (RQ1)
L3 Dynamic Stress	100-agent emulated mesh + RCU race	400 delegation chains + 250 bursts	RCU 420 ns pointer transition; 0 replayed executions	0	PASS (RQ2)
L4 Meta-Verification	Negative controls + 100 mutants	5 injected flaws + 100 mutants	5/5 flaws caught; 100% mutation score over 100 mutants	0	PASS (RQ3)
L5 Oracle Falsification	9 Oracle torture vectors	450 mutation probes	$G = 0$ ghost syscalls within S_{obs} ; 0 audit-state drift	0	PASS (RQ4)
L6 Config Validation	Cross-OS/arch configuration check	7,500 automated probes	Validated policy semantics portability across x86/ARM/Win	0	PASS

A. Adversarial Coverage vs. Results Matrix

The adversarial campaign exercises 10 distinct attack dimensions: Identity (Ed25519 rejection), Capability (bitmask mismatch), Parameter (SHA-256 hash mismatch), Temporal (RCU pointer revocation), Delegation (Task Issuer binding), Interpreter (C-ABI gate interception), Concurrency (RCU lock-free isolation), Encoding (canonical bitmask matching), Topology (independent verification across 100-agent mesh), and Failure (atomic two-phase commit). Across all 10 dimensions, 0 counterexamples were observed.

B. Meta-Verification: Negative Controls and Mutation Score (RQ3)

To evaluate whether the evaluation framework detects known security regressions (RQ3), we constructed 5 sabotaged binaries with deliberately injected flaws and evaluated 100 instantiated mutants across 8 mutation classes (M1: bit widening, M2: identity substitution, M3: expiry bypass, M4: hash omission, M5: stale reads, M6: check removal, M7: fail-open branch, M8: audit suppression):

$$\text{Mutation Score} = \frac{\text{Killed Mutants}}{\text{Total Instantiated Mutants}} = \frac{100}{100} = 1.0 \quad (8)$$

The harness detected 5/5 sabotaged flaws and killed 100/100 mutants, confirming that the harness is not a permissive rubber stamp.

V. DECOUPLED MULTI-OBSERVER ORACLE & EMPIRICAL RESULTS

A. The Four-Stage Independent Ground-Truth Pipeline (RQ4)

To enforce the principle that “No Single Component May Act As Its Own Sole Witness,” DROS-VEP establishes a 4-stage decoupled pipeline ($O_I \rightarrow O_A \rightarrow O_E \rightarrow O_P$):

$$\text{Intent } (O_I) \longrightarrow \text{Auth } (O_A) \longrightarrow \text{Exec } (O_E) \longrightarrow \text{Effect } (O_P) \quad (9)$$

- 1) **Intent Oracle (O_I):** Records raw tool invocation payloads directly from the caller.
- 2) **Authorization Oracle (O_A):** PGM C-ABI gate evaluates bitmasks and logs enforcement decisions (DENY).
- 3) **Execution Oracle (O_E):** Decoupled kernel monitor confirms $G = 0$ ghost syscalls escape within instrumented scope.
- 4) **Physical Effect Oracle (O_P):** Out-of-band DB WAL, filesystem, and process-tree hash diffs verify $S_{\text{after}} \equiv S_{\text{before}}$ over S_{obs} .

B. Physical Side-Effect Containment Analysis ($\Delta I_{\text{physical}} - RQ1$)

$$\Delta I_{\text{physical}} = I_{\text{physical}}(B0) - I_{\text{physical}}(B1) = 1.0 - 0.0 = 1.0 \quad (10)$$

Empirical evaluation demonstrates 100% observed physical-effect suppression over the evaluated attack corpus.

C. Performance & Latency Overhead

Evaluated on AMD Ryzen 9 7950X / Ubuntu 24.04:

- **End-to-End Decision Latency (P50):** 26.1 μs (HTTP parse, DIT Ed25519 verify, bitmask check).
- **End-to-End Decision Latency (P99):** 31.4 μs .
- **RCU State-Pointer Swap Latency (T_{swap}):** 420 ns (memory barrier & atomic swap, isolated from network I/O).
- **L4 Fail-Closed Denial Path Latency:** < 500 ns.

VI. OPEN FALSIFICATION PROTOCOL & LIMITATIONS

A. Limitations & Epistemic Scope (Gap in Scope)

- 1) **Ontological Dependency:** DROS enforcement relies on accurate capability bitmask mapping in L2 DITs.
- 2) **Observation Boundary:** The zero-counterexample finding is strictly bounded to the experimentally instantiated space of 60,000+ probes and 100-agent emulated meshes, and does not constitute a priori proof across infinite attack universes.
- 3) **Portability vs. Native Kernel Boundary (Gap in Scope):** L6 evaluates policy configuration portability across platforms; native kernel-level enforcement remains subject to host OS MMU and kernel driver guarantees.

B. Public Counterexample Registry

We publish the replication package on GitHub and establish an immutable counterexample registry ([reproducibility/counterexamples/](#)) inviting external researchers to submit reproducible traces of unauthorized state transitions.

VII. CONCLUSION & DECLARATIONS

This paper presented and experimentally evaluated the DROS 4-Layer Runtime Substrate with Deterministic Execution Enforcement. By sinking execution boundaries to a controlled binary C-ABI / FFI authorization layer, DROS bridges the semantic-kernel paradox in agentic workflows. Across the instantiated evaluation space, no executable counterexamples were observed:

$$\forall x \in X_{\text{evaluated}}, \quad C_A(x) \wedge \neg C_E(x) \implies I_{\text{physical}}(x) = 0 \quad (\text{PASS}) \quad (11)$$

ACKNOWLEDGMENT & AI COLLABORATION DISCLOSURE

In accordance with IEEE 2024+ authorship guidelines and enterprise AI governance transparency principles, the authors declare that the novel security concepts, physical-layer architecture, 6-pillar framework, mathematical formalisms, and patent claims presented in this work were independently conceptualized, architected, and validated by Chun-Cheng (Jimmy) Chen.

REFERENCES

- [1] J. Chen, “DROS: A Four-Layer Deterministic Runtime Operation System Bridging the Agent-to-Execution Attribution Gap in Autonomous AI Workloads,” *Zenodo Technical Report*, DOI: [10.5281/zenodo.22092008](#), 2026.
- [2] J. Chen, “DROS Trilogy Reading Guide: An Agent Runtime Operation Substrate (Academic Version 3.0),” *Zenodo Technical Guide*, DOI: [10.5281/zenodo.22114036](#), 2026.
- [3] J. Chen, “DROS-PGM: Physical Guard Module with Sub-Microsecond C-ABI Binary Execution Boundary,” *Zenodo Research Report*, DOI: [10.5281/zenodo.21903687](#), 2026.
- [4] J. Chen, “DROS 6P Architectural Specification: Unified Trust, PKI, and Execution Governance,” *Zenodo Specification*, DOI: [10.5281/zenodo.21833970](#), 2026.
- [5] Strix Security Team, “Strix: Autonomous Multi-Agent AI Penetration Testing Framework (v1.5.3),” 2026. [Online]. Available: <https://strix.ai>
- [6] Microsoft, “Microsoft Agent Framework Documentation: Tool Calling and Execution Governance,” *Microsoft Learn*, 2025.
- [7] NVIDIA, “NeMo Guardrails: Programmable Guardrails for LLM Applications,” *NVIDIA Developer Documentation*, 2024.
- [8] European Parliament, “Artificial Intelligence Act (Regulation EU 2024/1689), Article 50: Transparency and Traceability of AI Systems,” *Official Journal of the European Union*, 2024.
- [9] MITRE Corporation, “ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems,” *MITRE ATLAS Knowledge Base*, 2026.
- [10] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications,” *OWASP Standard*, 2025.
- [11] P. E. McKenney, “Is Parallel Programming Hard, And, If So, What Can You Do About It? (Read-Copy Update Architecture),” *Linux Technology Center, IBM Operating Systems Review*, 2024.
- [12] W. Enck *et al.*, “TaintDroid: An Information-Flow Tracking System for Real-Time Privacy Monitoring on Smartphones,” *ACM Transactions on Computer Systems (TOCS)*, vol. 32, no. 2, pp. 1–32, 2014.
- [13] METR (Model Evaluation and Threat Research), “Evaluating Autonomous Capabilities in Frontier AI Models,” *METR Technical Research Standard*, 2025.
- [14] USENIX Security Symposium, “Artifact Evaluation Guidelines and Criteria,” *USENIX Association*, 2024.
- [15] IEEE S&P Editorial Board, “IEEE Symposium on Security and Privacy: Call for Papers and Submission Guidelines,” *IEEE Computer Society*, 2026.